Mini Java Lexical Analyzer

Phase 1 implementation report | Python 3.10+

The lexical analyzer reads a Mini Java program character by character, groups its characters into tokens, and records their positions. It skips whitespace and comments, builds an identifier symbol table, and reports lexical errors. All scanning rules are implemented manually without a lexer or parser library.

Tokens implemented

- **Keywords:** `CLASS`, `PUBLIC`, `STATIC`, `VOID`, `INT`, `FLOAT`, `BOOLEAN_TYPE`, `CHAR`, `STRING_TYPE`, `IF`, `ELSE`, `WHILE`, `FOR`, `RETURN`.
- **Identifiers:** `IDENTIFIER` starts with an ASCII letter or underscore and continues with letters, digits, or underscores. Names are case-sensitive.
- **Literals:** `NUMBER` represents integers and decimal fractions; `STRING` represents values in matching single or double quotes; `BOOLEAN` represents `true` and `false`.
- **Operators:** `MATHOP` covers `+ - * / %`; `LOGOP` covers `== != < <= > >=` and the additional logical operators `&& || !`; `ASSIGN` represents `=`.
- **Punctuation:** `LPAR (`, `RPAR )`, `LCUR {`, `RCUR }`, `LSQU [`, `RSQU ]`, `SEMICOLON ;`, `COMMA ,`, and `DOT .`.
- **End of input:** `EOF` marks the end of the token list for the next compiler phase.

The type keyword `String` produces `STRING_TYPE`, while quoted values produce `STRING`. The keyword `boolean` produces `BOOLEAN_TYPE` to distinguish it from Boolean values. The reserved-word list does not include `main`, so it is an `IDENTIFIER`, despite `MAIN` in the illustrative output. Signs are separate operators; numbers with exponents or suffixes are outside this subset.

Scanning and symbol table

The scanner tracks an index, line, and column. Small lookahead distinguishes comments from division and recognizes two-character operators before shorter ones. A complete word is read before checking whether it is a keyword. Strings preserve their quotes and backslash escapes. Comment markers inside strings remain ordinary text; block comments do not nest.

The symbol table is a dictionary mapping each distinct identifier to a stable ID starting at 1. Repeated names reuse their ID, and identifier tokens carry it in `symbol_id`. Keywords and literals do not enter the table. Each token also stores its type, lexeme, line, and column. Lines and columns start at 1; Windows and Unix newlines are supported, and a tab counts as one column.

Error handling

Every error contains a descriptive message, line number, and column number. Scanning continues where possible, and invalid fragments are omitted from the token list. A parser should check the error list before using recovered tokens.

- **Unknown symbol:** Report its position, skip it, and continue. For example, `int x = 10 @ 5;` reports the invalid `@` at line 1, column 12. A lone `&` or `|` is also invalid.
- **Invalid identifier:** Consume an entire digit-led name such as `1name` or `12_count` and report its starting position. No partial number or identifier is emitted.
- **Unclosed string:** Report the position of the opening quote. Discard the unfinished literal and resume at the next line, or stop at the end of the source.
- **Unclosed block comment:** Consume the remaining input and report where the opening `/*` appeared. The token list still ends with `EOF`.

Challenges faced and solutions

- **Overlapping symbols:** Comments are checked before division, and two-character operators before one-character operators. This keeps `//`, `/*`, `>=`, and `==` from being split incorrectly.
- **Recovering after an error:** Invalid digit-led names are consumed together. An unfinished string stops at a newline so later valid statements can still be scanned.
- **Names versus expressions:** `total-marks` is scanned as `total`, `-`, `marks`, because it can be subtraction. `student@id` reports `@` and preserves the surrounding names. Syntax checks belong to Phase 2.

Verification and use

Automated tests cover the token families, positions, case sensitivity, comment boundaries, escaped quotes, error recovery, symbol reuse, repeated scanning, and command-line behavior. The sample and additional valid inputs scan without errors; the other two input files demonstrate lexical errors.

```
python lexer.py inputs/sample.txt
python -m unittest -v
```

The command line prints tokens, symbols, and errors. Exit codes are 0 for a clean scan, 1 for lexical errors, and 2 for incorrect usage or a file-read failure. The importable `Lexer(source).scan()` interface returns tokens, the symbol dictionary, and errors for Phase 2. The lexer and tests require no third-party packages.